DFIR-EVAL / agent capability harness THEME

POST-COMPROMISE HOST FORENSICS & REMEDIATION

Measuring what an agent does after the breach.

A self-scoring evaluation for **post-compromise DFIR**: hand an agent a read-only forensic image and a partial alert feed, and score whether it independently finds what else happened, grounds every claim in evidence, and writes a verified remediation plan. Built to run in Harbor, scale to thousands of ranges, and double as an **RL reward function**.

TASKS BUILT **10**	GEMINI 3.6 FLASH **5** / 10 pass
JUDGE-FREE SCORING **~65%**	DESIGNED TO SCALE TO **3,120** ranges

01 — THE INSTRUMENT

One subdomain, chosen because it is genuinely unsolved.

We anchor on **SecRespond** (Alibaba-NLP, arXiv:2607.26791) — the only cybersecurity corpus released after March 2026 that is simultaneously agentic

knowledge-work, rubric-scored, and CLI-compatible. Of 15 screened datasets it was the only one to clear our bar. The measured capability decomposes into four probes that are **separable in scoring** — which is what makes it a diagnostic instrument, not a pass/fail gate.

PROBE	QUESTION	AXIS
P1 Scope expansion	What else should I look for?	`detection`
P2 Evidence grounding	Can I cite it?	`detection`
P3 Attribution	Where in the chain?	`detection`
P4 Closure	Fixed *and* verified?	`plan`

SELECTION SCORECARD

- **Post-March-2026** data — uncontaminated by pre-training
- **Agentic** — multi-step CLI investigation, not multiple-choice
- **Structured deliverable** — five markdown reports, not a short answer
- **Rubric-scored & Harbor-portable** — 6 of 7 requirements met

---

02 — THE PUBLISHED BASELINE

Frontier models fail this in one reproducible way: they follow the alert trail and stop.

SecRespond evaluates **23 frontier models — Gemini 3.6 Flash among them** — across 280 checkpoints on the OpenCode harness (three-judge averaged). The paper's headline numbers, quoted:

SECRESPOND RESULT (PAPER)	DETECTION	PLANNING
Models with a complete solve on **any** range		**0 / 23**

Best overall — Claude Opus 4.7	79.0%	65.7%
Weakest — MiniMax M2.5	52.4%	32.9%
Persistence vs. intrusion-entity detection	58.8%	75.4%
Agentless production scanner (persistence)	2.1%	

At the paper's own 70%-per-axis "strong" cutoff, current frontier performance sits in a **25–40% band**. Our working extract does not reproduce SecRespond's individual Gemini-3.6-Flash cell, so we anchor to this band and to our own measured result below. **Knowledge ≠ capability:** CAIBench (arXiv:2510.24317) finds models score 70–89% on security *knowledge* tests but 20–40% on multi-step attack & defence — the premise this whole programme rests on.

---

03 — WHY WE REGENERATE INSTEAD OF REUSING THE TASKS

The ground truth has to be born with the task.

The deterministic verifier tier needs a **machine-readable manifest** of every artifact, chain edge, and remediation step. Human labelling can't produce that at thousands-of-ranges scale, and **SecRespond ships no manifest** — so its tasks can't drive Tier 1 (~45% of scoring). We instead **generate** ranges from attack scripts that **emit their own ground truth** as they run. Regeneration and verifier design are the same project.

SecRespond then stays a clean, **held-out benchmark** — never in the training loop, so the parity numbers it anchors stay trustworthy.

> WHY NOT SECRESPOND'S OWN EVAL FRAMEWORK?
>
> No manifest → no deterministic tier; and every leading SOC benchmark "still treats the unit of evaluation as a question" (SentinelOne Labs). We keep the **artifact** as the unit and move ~65% of scoring off the judge — cheaper, faster, and non-degenerate for RL.

04 — HOW A TASK & ITS IMAGE ARE BUILT

One authored file.
Everything else compiles.

A **blueprint** is the only per-range human input. It declares the scenario's zones — the ATT&CK-annotated attack chain, the artifact map (with disk paths and alerted/silent flags), decoys, and alert coverage. From it, the generator emits the disk image, the security-product feed, the manifest, and the full templated Harbor package — with **no task-specific verifier code**.

1 · BLUEPRINT

Declare the zones

chain · artifacts · decoys · alert coverage · difficulty

↓

2 · GENERATE

Run the attack

provision VM → exploit over real protocols → snapshot → collect telemetry

↓

3 · EMIT

manifest.json

disk delta + /sas feed + ground-truth manifest, content-addressed

↓

4 · PACKAGE

Harbor task

task.toml · Dockerfile · instruction.md · verifier — all templated

Public sources feeding blueprints: **MITRE ATT&CK** (technique chains), the **NVD / CVE** corpus (vulnerability context), and the held-out **SecRespond** ranges for parity. The keystone is `manifest.json`: the automation already knows every file it wrote — emitting it converts checkpoint authoring from a human step into template rendering.

---

05 — SCALING TO THOUSANDS, HONESTLY

Coverage over combinatorics — then five prompts per range.

Naïve mixing yields ~350k "combinations," but most are **reasoning-isomorphic** (Log4Shell→webshell→cron and Struts→webshell→cron are the *same* investigation). So we sample a capability × difficulty grid and report the statistically meaningful count — **distinct reasoning signatures** — not the inflated one.

52	×	4	×	15	=	3,120
CAPABILITY ITEMS		DIFFICULTY LEVELS		INSTANCES		RANGES · ~200 SIGNATURES

→ **~15k** RL PROMPTS

Difficulty's primary dial is **alert coverage** (90→10%), served from the mock feed rather than baked into the disk — so one immutable range yields ~5 prompts at different difficulties, and an adaptive curriculum removes an alert when rollouts all succeed, restores one when they all fail.

The QA loop — every generated range must pass all gates

GATE	THRESHOLD
Manifest ↔ disk consistency	100%
Oracle (manifest → reports)	≥ 0.90
No-op (empty but plausible)	≤ 0.10

No-op (empty-but-plausible)	≤ 0.10
Live agent trial	`0 < s < 0.95`
Signature cap	≤ 15
Cheat trial (manifest unreachable)	pass

> **THE LOAD-BEARING GATE**
>
> A verbose, confident, *evidence-free* report scores non-trivially against any LLM judge. If you don't test for it, you measure prose style. **No-op ≤ 0.10 and oracle ≥ 0.90** is enforced before any model is scored — and it held on all 10 of our ranges through every verifier revision.

---

06 — THE VERIFIER

Three tiers, two axes, and gates that punish bluffing.

Each of the five reports is scored against the manifest along **two axes that are never summed** — detection and plan — because the diagnostic value is in the *gap* between them. Most scoring is deterministic Python; the LLM judge is a thin, optional top layer.

TIER 1	**Deterministic checks** Citation resolution, required files & sections, no-fabrication — pure Python against /disk & /sas	~45%
TIER 2	**Tree heuristics** Attack-chain graph-edit distance, remediation coverage & precedence, per-step verification	~20%
TIER 3	**LLM-as-judge (optional)** Report-quality rubric only — off by default; excluded from the pass gate	~35%

Two gates sit on top, both grounded in the literature on over-prediction (OrgForge-IT:

identical F1 hid a **0.021 vs 0.813** false-positive gap):

```
# structure gate — a run missing a deliverable is incomplete, not low-scoring
structure = 1 iff all five reports present & parse # fabrication gate — any invented citation zeroes detection (no reward for carpet-bombing) fabrication = 0 required # the scalar reward is a hard AND, never a blended sum reward = 1 iff detection ≥ 0.70 AND plan ≥ 0.70 AND structure=1
```

---

07 — AS A REWARD FUNCTION FOR RL FINE-TUNING

The same verifier trains the model it grades.

The binary pass is only the **outcome** signal. For GRPO, the verifier also emits a **dense, per-checkpoint** score vector — the real training input — so gradient survives even when the outcome term is flat. Every rollout carries a multiplicative **gate**: fabricating or unsafe trajectories earn zero advantage from either path.

Dense advantage per checkpoint beats one scalar — zero-variance groups are ~40% of agentic RL

~65% judge-free scoring keeps rollout cost sane at group size 8

Closed-weight fallbacks: **GEPA** scaffold optimization or reject-sampling SFT on high-reward traces

WHY THE REWARD IS SHAPED THIS WAY — THE EVIDENCE

Hard-zero on fabrication — rubric judges reward verbosity; unchecked, RL learns to carpet-bomb

Detection ≠ plan, never summed — SecRespond's dataset card mandates it; the gap is the signal

Precision as first-class — OrgForge-IT & frontier-readiness studies on FP collapse

Don't trust self-reported confidence — ReCon found it was pure verdict-restatement

08 — HARBOR TASK-QUALITY CHECK

Passing every core-quality gate.

Harbor's automated reviewer graded each task against its quality rubric. The gates that **passed across the suite** (packaging-metadata and separate-verifier items omitted):

- ✓ **Verifiable**
- ✓ **Solvable** (oracle 1.0)
- ✓ **Difficult**